Nile OS RTOS

Instruction manual

Date: 03.09.2026
Kernel version: 1.2.1
Manual version: 1.0.0

OS Manual Change Log

Version	Changes
1.0.0	First release

Introduction

Nile OS (simply “Nile” or “OS” from now on) is intended to be a highly-flexible RTOS with a lot of knobs to configure it for a specific use case. While it could be classified as a hobby project, the ultimate goal is a full-featured robust operating system.

The OS logic is separated into platform-agnostic level and hardware-specific parts, which hook up into platform-agnostic logic where required. This should make porting the OS to other architectures (such as RISC-V) easy.

OS Features

Highly flexible scheduling with a lot of parameters to create various scheduling patterns from fair round-robins, to all sorts of unfair ones, including mixed patterns for specific tasks.

Memory protection: the system provides a configurable number of memory protection regions for tasks to use, the number of memory protection regions per task is set globally and is the same for all tasks. Tasks don’t have to use all of them. Tasks need at the very least access to their own code section and access to their own stack space.

Kernel provides its own TLSFA (Two-level segregated fit, aligned) fragmentation-resistant allocator with O(1) insertion and O(1) deletion. The allocator is used to provide memory for idle task stack space (minimal), IO device operation buffers, and other optional structures.

IO devices are grouped into character devices and block devices. All IO operations work directly on task memory, without any intermediary kernel buffers involved (e.g. UART read request receives data straight into task memory). Tasks access IO via system calls. Tasks can spin on IO, can block on IO, or block on IO with a timeout (upon which the IO operation is cancelled, and if the task was blocked, the task is woken up with an indication of a timeout).

Scheduler

WARNING: Task priority grows in ASCENDING order. 0 is the lowest priority.
WARNING: OS interrupt priority grows in ASCENDING order. 0 is the lowest priority. Priority 0 is reserved for context switch interrupt, priority 1 is reserved for OS tick source, priority 2 is reserved for system calls. Priority 3 and higher can be used for other hardware interrupts. Maximum priority level (hardware-dependent)

is reserved for system exceptions (invalid instruction, memory protection violation, etc.).

The OS has an OS tick prescaler – how many hardware ticks it takes for a single OS tick to happen. Everything else in the system is timed using OS ticks.

Tasks have base priority level and dynamic priority level, tasks can increase their priority by a configurable amount per configurable number of OS ticks, individual per task. Each task also has a burst length – a number of ticks the task runs without being switched out (unless the task voluntarily yields/blocks).

The scheduler has two task queues: a ready queue and a blocked queue. Each of them is a doubly linked list of Task Control Blocks, which contain tasks' scheduling parameters, as well as saved context.

The ready queue is sorted by priority, the highest priority is first in the queue, and it will be the next task to execute. If ready queue is empty, idle task is scheduled to run.

The blocked queue consists of 3 regions (implicitly defined by tasks' blocked flags): Region 1 is IO blocked tasks (no timeout), Region 2 is IO blocked tasks with timeouts in ascending wakeup order, followed by Region 3 of Timeout blocked tasks (no IO) in descending wakeup order. Unblocking routine checks all IO blocked tasks for an IO event to wake them up (Region 1), and checks IO events and timeout events (timeouts only until the first task that's not timed out – all the next tasks' timeout events are even later), and then it traverses Region 3 timed out tasks in reverse order until the first task that's not timed out (same logic as Region 2, but with traversal going back the through the list).

Tasks can be RUNNING (executing now), READY (in ready queue), BLOCKED (in blocked queue, not being scheduled).

OS takes care of a potential OS tick counter overflow, and, upon reaching a critical timestamp, the OS tick counter and all wakeup timestamps of blocked tasks are updated accordingly (reset to low values).

IO Devices

All IO devices are either character devices or block devices, and the number of device slots of each category is set at compile time, and the devices are initialized during the OS initialization. Devices have function pointer hooks to hardware-functions, namely init routine, a reset routine (which is often similar or identical to init routine), as well as start and stop routines for operations. Block devices also have hooks for enabling and disabling memory-mapped mode (which is the preferred way for read operations, if applicable). Devices can report device-specific error codes to tasks (which wakes the tasks up from blocked state similarly to finished IO operations, but with an error code). Each device has 7 bits of error code space.
Devices can be shared, the shared flag is set (or not) during kernel device initialization. A device that is not shared, will have to be opened by a task using the device (with a system call), otherwise the IO operation will be denied and the IO request system call will return an error code. The task can close the opened device via another system call to make it openable by another task. Shared devices can be accessed by all tasks at any time without opening/closing.
The OS contains pipes as simulated character devices for inter-process communication, where kernel manually copies data from one task's TX buffer to another task's RX buffer (as soon as there is a TX request and RX request to the same pipe). Pipes should be initialized as shared devices.

System calls

The OS supports multiple system calls, which can be easily accessed via a unified system call interface. The user task calls the system call function with 3 arguments: system call number, pointer to system call argument array of 32-bit values, and a number of arguments in that array (number of 32-bit values). System calls may use that argument structure as input parameters of a system call, as well as outputs or output parameters, depending on system call. System call requests return a 32-bit system call request code, which either contains 0 (NILE_SYSCALL_RETVAL_OK), or a non-zero error code, which will specify the problem with a system call request (e.g. bad argument count, or task requested IO read into memory outside the task's writeable memory protection regions).

IO system calls (read/write/erase requests, as well as IOCTL) use a unified IO system call structure as a system call input parameter (regardless of device type and operation type). All IO system calls themselves are not blocking and return immediately. Should the user want to block on the IO operation, they should use a blocking system call after the IO system call.

Task blocking system call uses a unified blocking structure to block the task (if the blocking has IO, it contains a pointer to the IO structure).

System call table.

System call	Arg list	Behavior
NILE_SYSCALL_KERNEL_INFO_VERSION	Args: Arg[0]: Pointer to output 32-bit element array (Out) Arg[1]: Output count (32-bit values)	Outputs the following into output array: Out[0] = kernel size (bytes) Out[1] = kernel heap space size (bytes) Out[2] = kernel version major Out[3] = kernel version minor Out[4] = kernel version patch If output count is less than 5, outputs only the values that fit into the output array (e.g. for output count 1, outputs only kernel size)
NILE_SYSCALL_KERNEL_INFO_TIMING	Args: Arg[0]: Pointer to output 32-bit element array (Out) Arg[1]: Output count (32-bit values)	Outputs the following into output array: Out[0] = OS tick duration in microseconds Out[1] = OS tick hardware timer frequency Out[2] = OS tick prescaler If output count is less than 3, outputs only the values that fit into the output array (e.g. for output count 1, outputs only OS tick duration)
NILE_SYSCALL_TASK_YIELD	Don’t care	The task stops execution immediately as its execution burst is considered finished, and it gets reinserted into the scheduler’s ready queue. The OS immediately switches to idle task until the next OS tick.
NILE_SYSCALL_TASK_BLOCK	Args: Arg[0]: Pointer to nile_syscall_params_task_block structure	Members of the structure are: 1. block_type (IO, timeout, IO with timeout)

	Arg[1]: blocking parameters list length (use NILE_SYSCALL_PARAM_CNT_TASK_BLOCK)	2. pointer to IO structure (don't care if non-IO block; IO system call must be executed before blocking on IO) 3. Timer value – for timed blocking, depending on blocking type it's either microseconds or milliseconds 4. Return value block release source – must be zero at the moment of blocking system call, once the task wakes up, the field will contain a value of wake up source (IO wake up or timeout wake up). If unblocking event happened before blocking system call was called (e.g. IO operation completed quickly), the system call is no-op and returns straight into the task.
NILE_SYSCALL_IO_CHAR_DEV_OPEN NILE_SYSCALL_IO_CHAR_DEV_CLOSE NILE_SYSCALL_IO_BLOCK_DEV_OPEN NILE_SYSCALL_IO_BLOCK_DEV_CLOSE NILE_SYSCALL_IO_CHAR_DEV_READ NILE_SYSCALL_IO_CHAR_DEV_WRITE NILE_SYSCALL_IO_BLOCK_DEV_READ NILE_SYSCALL_IO_BLOCK_DEV_WRITE NILE_SYSCALL_IO_BLOCK_DEV_ERASE	Args: Arg[0]: Pointer to nile_syscall_params_io_op structure Arg[1]: IO parameters list length (use NILE_SYSCALL_PARAM_CNT_IO_OP)	Members of the structure are: 1. Device ID (char device number or block device number) 2. Pointer to data buffer (32-bit aligned) 3. Data buffer byte length 4. Operation flags (additional hw-specific IO parameters) 5. Block device operation offset 6. Return value for potential blocking 7. Return value for potential blocking 8. Return value for potential blocking

		9. Return value for potential blocking
WARNING! NOT IMPLEMENTED YET. NILE_SYSCALL_IO_CHAR_DEV_IOCTL NILE_SYSCALL_IO_BLOCK_DEV_IOCTL	Args: Arg[0]: Pointer to nile_syscall_params_io_op structure (same data structure as for IO operations, fields reused) Arg[1]: IO parameters list length (use NILE_SYSCALL_PARAM_CNT_IO_OP)	Members of the structure are: 1. Device ID (char device number or block device number) 2. Pointer to IOCTL data buffer 3. IOCTL data buffer byte length 4. IOCTL command code 5. Unused 6. IOCTL arg0 7. IOCTL arg1 8. IOCTL arg2 9. IOCTL arg3

Kernel Initialization

Kernel is initialized over zeroed out memory, with kernel heap memory following the kernel structure. Task control blocks are allocated in kernel heap. Idle task also allocates its stack space in kernel heap.

Task initialization routines allocate task control blocks in kernel heap, with the appropriately sized structure which may or may not include space for saving FPU context. Allocated TCB must be zeroed out.

The task needs to have scheduling parameters set, memory protection regions configured (region size set to 0 means unused region), its stack space and stack space size set, FPU configuration set (FPU used/not used, FPU lazy stacking used/not used).

Then the startup frame of the task must be created (all register values at task startup). Finally, the task is added to the ready queue.

Example

Use the provided example code for initialization and configuration procedures, task and device initialization, as well as for system calls use.